

Лабораторная работа № 8. Оптимизация

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

Информация

- Абакумова Олеся Максимовна
- Студентка
- Российский университет дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

Выполнение лабораторной работы

Под оптимизацией в математике и информатике понимается решение задачи нахождения экстремума (минимума или максимума) целевой функции в некоторой области конечномерного векторного пространства, ограниченной набором линейных и/или нелинейных равенств и/или неравенств.

Оптимизационной задачей называется задача определения наилучших с точки зрения структуры или значений параметров объектов.

В Julia для обработки данных используются наработки из других языков программирования, в частности, из R и Python.

Линейное программирование рассматривает решения экстремальных задач на множествах n -мерного векторного пространства, задаваемых системами линейных уравнений и неравенств.

Общей (стандартной) задачей линейного программирования называется задача нахождения минимума линейной целевой функции вида:

$$f(\vec{x}) = \sum_{j=1}^n c_j x_j,$$

где $\vec{c}$ — коэффициенты, $\vec{x} \in \mathbb{R}^n$.

Основной задачей линейного программирования называется задача, в которой есть ограничения в форме неравенств:

$$\sum_{j=1}^n a_{ij}x_j \geq b_i, \quad i = 1, 2, \dots, m,$$

$$x_j \geq 0, \quad j = 1, 2, \dots, n.$$

Задачи линейного программирования со смешанными ограничениями, такими как равенства и неравенства, с наличием переменных, свободных от ограничений, могут быть сведены к эквивалентным с тем же множеством решений путём замены переменных и замены равенств на пару неравенств.

JuMP включает пакет `Convex.jl` (<https://jump.dev/Convex.jl/stable/>), позволяющий описать задачу оптимизации, используя естественный математический синтаксис, и решать её с помощью одного из решателей (COSMO, ECOS, SCS, GLPK, MathOptInterface). Предположим, что требуется решить следующую задачу линейного программирования:

$$12x + 20y \rightarrow \min$$

при ограничениях:

$$6x + 8y \geq 100,$$

$$7x + 12y \geq 120$$

Воспользуемся JuMP и решателем линейного и смешанного целочисленного программирования GLPK:

```
#  
import Pkg  
Pkg.add("JuMP")  
Pkg.add("GLPK")  
using JuMP  
using GLPK
```

```
]# Определение объекта модели с именем model:  
model = Model(GLPK.Optimizer)  
  
] A JuMP Model  
├ solver: GLPK  
├ objective_sense: FEASIBILITY_SENSE  
├ num_variables: 0  
├ num_constraints: 0  
└ Names registered in the model: none  
  
]# Определение переменных x, y и граничных условий для них:  
@variable(model, x >= 0)  
@variable(model, y >= 0)
```

Рис. 1: Определение объекта модели и определение граничных усл. для переменных

```
: # Определение ограничений модели:
@constraint(model, 6x + 8y >= 100)
@constraint(model, 7x + 12y >= 120)

: 7x + 12y ≥ 120

: # Определение целевой функции:
@objective(model, Min, 12x + 20y)

: 12x + 20y

: # Вызов функции оптимизации:
optimize!(model)

: # Определение причины завершения работы оптимизатора:
termination_status(model)

: OPTIMAL::TerminationStatusCode = 1

: # Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результата оптимизации:
@show objective_value(model);

value(x) = 14.999999999999993
value(y) = 1.25000000000000047
objective_value(model) = 205.0
```

Рис. 2: Определение модели, цел. функции, ее вызов, статус работы оптимизатора и результат

Векторизованные ограничения и целевая функция оптимизации

Часто бывает полезно создавать коллекции переменных JuMP внутри более сложных структур данных. Можно добавить ограничения и цель в JuMP, используя векторизованную линейную алгебру.

Предположим, что требуется решить следующую задачу:

$$c^T \vec{x} \rightarrow \min$$

при заданных ограничениях:

$$A\vec{x} = \vec{b}, \quad \vec{x} \geq 0, \quad \vec{x} \in \mathbb{R}^n,$$

где:

$$A = \begin{pmatrix} 1 & 1 & 9 & 5 \\ 3 & 5 & 0 & 8 \\ 2 & 0 & 6 & 13 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 7 \\ 3 \\ 5 \end{pmatrix}, \quad \vec{c} = \begin{pmatrix} 1 \\ 3 \\ 5 \\ 2 \end{pmatrix}.$$

```
#  
import Pkg  
Pkg.add("JuMP")  
Pkg.add("GLPK")  
using JuMP  
using GLPK
```

Векторизованные ограничения и целевая функция оптимизации

```
: # Определение объекта модели с именем vector_model:
vector_model = Model(GLPK.Optimizer)

: A JuMP Model
  | solver: GLPK
  | objective_sense: FEASIBILITY_SENSE
  | num_variables: 0
  | num_constraints: 0
  | Names registered in the model: none

: # Определение начальных данных:
A = [ 1 1 9 5;
      3 5 0 8;
      2 0 6 13]
b = [7; 3; 5]
c = [1; 3; 5; 2]

: 4-element Vector{Int64}:
 1
 3
 5
 2

: # Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0)

: 4-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]
 x[4]
```

Рис. 3: Определение объекта модели, начальных данных и вектора переменных 16/59

Векторизованные ограничения и целевая функция оптимизации

```
] : # Определение ограничений модели:
    @constraint(vector_model, A * x .== b)

] : 3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintInterface.EqualTo{Float64}}, ScalarShape{}}:
      x[1] + x[2] + 9 x[3] + 5 x[4] == 7
      3 x[1] + 5 x[2] + 8 x[4] == 3
      2 x[1] + 6 x[3] + 13 x[4] == 5

] : # Определение целевой функции:
    @objective(vector_model, Min, c' * x)

] :  $x_1 + 3x_2 + 5x_3 + 2x_4$ 

] : # Вызов функции оптимизации:
    optimize!(vector_model)

] : # Определение причины завершения работы оптимизатора:
    termination_status(vector_model)

] : OPTIMAL::TerminationStatusCode = 1

] : # Демонстрация результата оптимизации:
    @show objective_value(vector_model);

    objective_value(vector_model) = 4.9230769230769225
```

Рис. 4: Определение модели, функции, вызов статус работы оптимизатора и результат

Оптимизация рациона питания

В некоторых задачах требуется использование массивов, в которых индексы не являются целыми диапазонами с отсчётом от единицы. Например, требуется использовать переменную, индексируемую по названию продукта или местоположению. Тогда необходимо в качестве индекса использовать произвольный вектор. В Julia это можно реализовать с помощью **DenseAxisArrays**.

Рассмотрим применение **DenseAxisArrays** на примере решения задачи оптимизации рациона питания в заведении быстрого питания при условии, что задано ограничение на количество потребляемых калорий (1800–2200), белков (≥ 91), жиров (0–65) и соли (0–1779), а также перечень определённых продуктов питания с указанием их стоимости

продукт	калории	белки	жиры	соль
гамбургер	10	24	26	730
курица	420	32	10	1190
сосиска в тесте	560	20	32	1800
жареный картофель	380	4	19	270
макароны	320	12	10	930
пицца	320	15	12	820
салат	320	31	12	1230
молочный коктейль	100	8	2.5	125
мороженное	330	8	10	180

Рис. 5: Содержание калорий, белков, жиров и соли в продуктах питания

```
#  
:  
import Pkg  
Pkg.add("JuMP")  
Pkg.add("GLPK")  
using JuMP  
using GLPK
```

Оптимизация рациона питания

```
: # Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray{
    [1800 2200;
     0 65;
     0 65;
     0 1779],
    ["calories", "protein", "fat", "sodium"],
    ["min", "max"]}

: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
  Dimension 2, ["min", "max"]
And data, a 4x2 Matrix{Float64}:
1800.0  2200.0
  0.0    65.0
  0.0    65.0
  0.0  1779.0

: # Массив данных с наименованиями продуктов:
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]

: 9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"

: # Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
    [2.40, 2.80, 1.50, 1.80, 2.00, 1.90, 2.40, 0.80, 1.50],
    foods}

: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.40
 2.80
 1.50
```

Рис. 6: Контейнер для хранения данных, массив данных с именами продуктов и массив стоимости продуктов

Оптимизация рациона питания

```
# Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
food_data = JuMP.Containers.DenseAxisArray{
  [410 24 26 730;
  420 32 10 1190;
  560 20 32 1800;
  300 4 19 270;
  320 12 10 930;
  320 15 12 820;
  320 31 12 1230;
  100 8 2.5 125;
  330 8 10 180],
  foods,
  ["calories", "protein", "fat", "sodium"]}

# 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
 410.0  24.0  26.0  730.0
 420.0  32.0  10.0 1190.0
 560.0  20.0  32.0 1800.0
 300.0   4.0  19.0  270.0
 320.0  12.0  10.0  930.0
 320.0  15.0  12.0  820.0
 320.0  31.0  12.0 1230.0
 100.0   8.0   2.5  125.0
 330.0   8.0  10.0  180.0

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

# A JuMP Model
 solver: GLPK
 objective sense: FEASIBILITY_SENSE
 num_variables: 0
 num_constraints: 0
 Names registered in the model: none

# Определение массива:
categories = ["calories", "protein", "fat", "sodium"]

# 4-element Vector{String}:
 "calories"
 "protein"
 "fat"
 "sodium"
```

AK
ЧК

Рис. 7: Массив данных о содержании КБЖУ, определение объекта модели и поределение массива с категориями

Оптимизация рациона питания

```
] # Определяем переменные:
@variables(model, begin
  category_data[c, "min"] <= nutrition[c - categories] <= category_data[c, "max"]
  # Число калорий - polynomial
  buy[foods] >= 0
end)

:] (1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
  nutrition[calories]
  nutrition[protein]
  nutrition[fat]
  nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{VariableRef}:
  buy[hamburger]
  buy[chicken]
  buy[hot dog]
  buy[fries]
  buy[macaroni]
  buy[pizza]
  buy[salad]
  buy[milk]
  buy[ice cream])

:] # Определяем целевую функцию:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))

:] 2.49buy_hamburger + 2.89buy_chicken + 1.5buy_hotdog + 1.89buy_fries + 2.09buy_macaroni + 1.99buy_pizza + 2.49buy_salad + 0.89buy_milk + 1.59buy_icecream

:] # Определяем ограничения модели:
@constraint(model, [c in categories],
  sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])

:] (1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1,...}} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}, ScalarShape{1,...}}:
  -nutrition[calories] + 400 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] == 3300
  -nutrition[protein] + 320 buy[salad] + 100 buy[milk] + 330 buy[ice cream] == 0
  -nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza]
```

Рис. 8: Определение переменных, целевой функции и ограничений модели

```
# Вызов функции оптимизации:  
JuMP.optimize!(model)  
term_status = JuMP.termination_status(model)
```

```
OPTIMAL::TerminationStatusCode = 1
```

```
hcat(buy.data, JuMP.value.(buy.data))
```

```
9x2 Matrix{AffExpr}:  
 buy[hamburger]  0.6045138888888888  
 buy[chicken]    0  
 buy[hot dog]    0  
 buy[fries]      0  
 buy[macaroni]   0  
 buy[pizza]      0  
 buy[salad]      0  
 buy[milk]       6.9701388888888935  
 buy[ice cream]  2.5913194444444441
```

Рис. 9: Вызов функции оптимизации и результат

Рассмотрим решение задачи определения оптимального числа паспортов, требующихся, чтобы объехать весь мир. При этом будем учитывать сведения о паспортах и ограничениях, указанных на ресурсе <https://www.passportindex.org/>. В табличной форме данные можно получить с ресурса <https://github.com/ilyankou/passport-index-dataset>. Для решения задачи представляют интерес данные, указанные в файле `passport-index-matrix.csv`, в котором в первом столбце (Passport) указана страна отбытия (= от), в остальных столбцах указаны страны назначения (= до), на пересечении строк и столбцов указаны условия по наличию или отсутствию визы или другие ограничения.

Значение	Пояснение
7–360	Количество безвизовых дней (при наличии)
VF	viza free (безвизовый режим)
VOA	viza on arriva (виза по прибытии)
ETA	e-viza или electronic travel authority (электронная виза)
VR	viza required (требуется виза)
covid ban	запрет в связи с COVID=19
no admission	въезд запрещён
-1	паспорт из места назначения

Рис. 10: Обозначения в сводной матрице по паспортам

```
#  
import Pkg  
Pkg.add("DelimitedFiles")  
Pkg.add("CSV")  
using DelimitedFiles  
using CSV
```

```
passportdata = readlm(joinpath("passport-index-dataset", "C:/Users/user/Downloads/passport-index-matrix.csv"),',',')

200x200 Matrix{Any}:
"Passport"      "Albania"      -   "Afghanistan"
"Afghanistan"   "e-visa"       -1
"Albania"       -1             "visa required"
"Algeria"       "e-visa"       "visa required"
"Andorra"       90             "visa required"
"Angola"        "e-visa"       "visa required"
"Antigua and Barbuda" 90             "visa required"
"Argentina"     90             "visa required"
"Armenia"       90             "visa required"
"Australia"     90             "visa required"
"Austria"       90             "visa required"
"Azerbaijan"    90             "visa required"
"Bahamas"       90             "visa required"
|
"United Arab Emirates" 90             "visa required"
"United Kingdom"      90             "visa required"
"United States"        360            "visa required"
"Uruguay"              90             "visa required"
"Uzbekistan"           "e-visa"       "visa required"
"Vanuatu"              "e-visa"       "visa required"
"Vatican"             90             "visa required"
"Venezuela"           90             "visa required"
"Vietnam"             "e-visa"       "visa required"
"Yemen"               "e-visa"       "visa required"
"Zambia"              "e-visa"       "visa required"
"Zimbabwe"            "e-visa"       "visa required"

# Задача переименования:
cntr = passportdata[2:end,1]
vf = {x -> typeof(x) == Int64 || x == "VF" || x == "VDA" ? 1 : 0}.(passportdata[2:end,2:end]);

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}()

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ names registered in the model: none
```

Ак
4тс

Рис. 11: Чтение файла, задание переменных и определение объекта модели

```
# Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(entr)], Bin)
@constraint(model, [j:1:length(entr)], sum{ v in 1:length(entr)} pass[v] for i in 1:length(entr)) >= 1)
@objective(model, Min, sum(pass))

pass1 + pass2 + pass3 + pass4 + pass5 + pass6 + pass7 + pass8 + pass9 + pass10 + pass11 + pass12 + pass13 + pass14 + pass15 + pass16 +
pass17 + pass18 + pass19 + pass20 + pass21 + pass22 + pass23 + pass24 + pass25 + pass26 + pass27 + pass28 + pass29 + pass30 +
+ [...] 139 terms omitted [...] + pass139 + pass140 + pass141 + pass142 + pass143 + pass144 + pass145 + pass146 + pass147 + pass148 + pass149 + pass150 + pass151 + pass152 + pass153 +
+ pass154 + pass155 + pass156 + pass157 + pass158 + pass159

# Run the dynamic programming:
jump_optimize(model)
termination_status(model)

OPTIMAL::TerminationStatusCode = 1

# Печатаем результаты:
print(jump_objective_value(model), " passports: ", join(entr[findall(jump_value.(pass) .== 1)], ", ", ""))

34.0 passports: Afghanistan, Australia, Bahrain, Cameroon, Comoros, Congo, Djibouti, Dominican Republic, Eritrea, Guinea-Bissau, Hong Kong, Iran, Kenya, Kuwait, Liberia, Libya, Madagascar, Maldives, Mauritania, Morocco, Nepal, New Zealand, North Korea,
Palestine, Papua New Guinea, Qatar, Saudi Arabia, Singapore, Somalia, South Sudan, Spain, Syria, Turkmenistan, United States
```

Рис. 12: Переменные, ограничения и целевая функция. Вызов функции и результат

Предположим требуется решить оптимизационную задачу в следующей формулировке. Имеется капитал в 1000 ден. ед., который планируется инвестировать в три компании — Microsoft, Facebook, Apple. При этом есть данные еженедельных значений цен на акции этих компаний за определённый период времени. Необходимо определить доходность акций каждой компании за рассматриваемый период времени, после чего инвестировать в эти три компании так, чтобы получить возврат в размере не менее 2% от вложенной суммы.

Для решения оптимизационной задачи будем использовать пакет `Convex.jl` и оптимизатор (решатель) `SCS`. Кроме того, понадобится пакет `Statistics.jl` для получения матрицы рисков на основе расчёта ковариационных значений по доходности.

Сначала требуется подгрузить имеющиеся данные о еженедельных значениях цен на акции рассматриваемых компаний, отобразить данные на графике. Предположим данные размещены в файле `stock_prices.xlsx` в каталоге `data` в проекте.

```
#  
using DataFrames  
using XLSX  
using Plots  
pyplot()  
using Convex  
using SCS  
using Statistics
```



```
T = DataFrame(XLSX.readtable("Downloads/stock_prices.xlsx", "Sheet2"))
```

13x3 DataFrame

Row	MSFT	FB	AAPL
	Any	Any	Any
1	101.93	137.95	148.26
2	102.8	143.8	152.29
3	107.71	150.04	156.82
4	107.17	149.01	157.76
5	102.78	165.71	166.52
6	105.67	167.33	170.41
7	108.22	162.5	170.42
8	110.97	161.89	172.97
9	112.53	162.28	174.97
10	110.51	169.6	172.91
11	115.91	165.98	186.12
12	117.05	164.34	191.05
13	117.94	166.69	189.95

4

Рис. 13: Подгружение данных

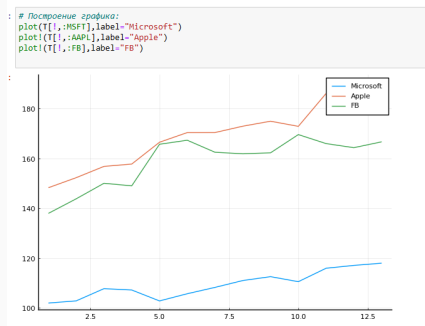


Рис. 14: Изменение цен на акции компаний за 13 дней

Для дальнейших расчётов данные по ценам на акции переформируем из фрейма в матрицу. Следует опираться на следующие формулы:

1. Доходность акций:

$$R(i, t) = \frac{pr(i, t) - pr(i, t - 1)}{pr(i, t - 1)}.$$

2. Целевая функция (минимизация риска):

$$\vec{x}^T \cdot \text{cov}(R) \cdot \vec{x} \rightarrow \min .$$

3. Ограничения:

Портфельные инвестиции

```
# Данные о ценах на акции размещаем в матрице:
prices_matrix = Matrix(T)

13x3 Matrix[Any]:
101.93  137.95  148.26
102.8   143.8   152.29
107.71  150.04  156.82
107.17  149.01  157.76
102.78  165.71  166.52
105.67  167.33  170.41
108.22  162.5   170.42
110.97  161.89  172.07
112.53  162.28  174.97
110.51  169.6   172.91
115.91  165.98  186.12
117.05  164.34  191.05
117.94  166.69  189.95

# Вычисление матрицы доходности за период времени:
M1 = prices_matrix[1:end-1,:]
M2 = prices_matrix[2:end,:]
# Матрица доходности:
R = (M2.-M1)./M1

12x3 Matrix[Float64]:
 0.00853527  0.0424067  0.027182
 0.0477626  0.0433936  0.0297459
-0.005081346 -0.00686484  0.00599413
-0.040963  0.112073  0.055274
 0.0281183  0.00977611  0.0233606
 0.0241317 -0.0288651  5.8682e-5
 0.0254112 -0.00375385  0.014963
 0.0140579  0.00240904  0.0115627
-0.0179508  0.0451072  -0.0117734
 0.0488644 -0.0213443  0.0763981
 0.00983522 -0.00988071  0.0264883
 0.00760359  0.0142996  -0.00575766

# Матрица рисков:
risk_matrix = cov(R)
# Проверка положительной определенности матрицы рисков:
Isposdef(risk_matrix)

true
```

Рис. 15: Данные о ценах на акции в матрице, вычисление матрицы доходности, матрица рисков и проверка положительной определенности матрицы рисков

Портфельные инвестиции

Доход от каждой из компаний получим из матрицы доходности как вектор средних значений. Далее нужно собственно сформулировать оптимизационную задачу. Пусть вектор $\vec{x} = (x_1, x_2, x_3)$ — вектор инвестиций, вектор $\vec{r} = (r_1, r_2, r_3)$ — вектор доходов от компаний. Тогда задача оптимизации будет иметь вид:

$$\vec{x}^T \text{cov}(R) \vec{x} \rightarrow \min$$

при условии:

$$\sum_{i=1}^3 x_i = 1, \quad \text{dot}(\vec{r}, \vec{x}) \geq 0,02, \quad x_i \geq 0, \quad i = 1, 2, 3,$$

где $\text{dot}(\vec{r}, \vec{x})$ — функция, определяющая возврат инвестиций. Формируем вектор инвестиций и определяем объект модели в соответствии с формулировкой оптимизационной задачи (при этом делаем задачу совместимой с DCP в соответствии с требованием пакета **Convex.jl**)

Портфельные инвестиции

```
: # Доход от каждой из компаний:
r = mean(R,dims=1)[: ]

: 3-element Vector{Float64}:
 0.012532748705136572
 0.016563036855293173
 0.02114580465503291

: # Вектор инвестиций:
x = Variable(length(r))

: Variable
 size: (3, 1)
 sign: real
 vexity: affine
 id: 385_610

: # Объект модели:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])

: Problem statistics
  problem is DCP      : true
  number of variables : 1 (3 scalar elements)
  number of constraints : 5 (5 scalar elements)
  number of coefficients : 19
  number of atoms      : 14

: Solution summary
  termination status : OPTIMIZE_NOT_CALLED
  primal status      : NO_SOLUTION
  dual status        : NO_SOLUTION
```

Рис. 16: Доход от каждой из компаний, вектор инвестиций, объект модели

```
# Minimize portfolio:
solve(problem, SCS.Optimizer)

-----
SCS v3.2.0 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 12
cones:   z: primal zero / dual free vars: 1
         1: linear vars: 4
         q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho: 1.00e-00
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
          nnz(A): 22, nnz(P): 0
-----
iter | pri res | dual res | gap | obj | scale | time (s)
-----
0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 2.97e-03
75 | 1.70e-04 | 3.73e-05 | 5.25e-05 | 4.62e-04 | 1.00e-01 | 3.15e-03
-----
status: solved
timings: total: 3.36e-03s = setup: 2.23e-04s + solve: 3.13e-03s
         lin-sys: 3.35e-05s, cones: 3.17e-05s, accel: 4.90e-06s
-----
objective = 0.000462
-----
[ Info: [Convex.jl] Compilation finished: 13.07 seconds, 916.313 MiB of m
-----
: Problem statistics
  problem is DCP      : true
  number of variables : 5 (3 scalar elements)
  number of constraints : 5 (5 scalar elements)
  number of coefficients : 19
  number of atoms      : 14

Solution summary
  termination status : OPTIMAL
  primal status      : FEASIBLE_POINT
  dual status        : FEASIBLE_POINT
  objective value     : 0.0005
```

Рис. 17: Нахождение решения

Выводим значения компонент вектора инвестиций и проверяем выполнение условия $\sum_{i=1}^3 x_i = 1$. Также проверяем выполнение условия на уровень доходности от 2%, переводим процентные значения компонент вектора инвестиций в фактические денежные единицы. Далее сформируем новый фрейм, включающий исходные данные о недвижимости и столбец с данными о назначенном каждому дому кластере.

```
: x
: Variable
  size: (3, 1)
  sign: real
  vexity: affine
  id: 385...610
  value: [0.07740709795031048, 0.11606771003983767, 0.8065276266877964]

: sum(x.value)
: 1.0000024346779446

: r'*x.value
: 1x1 adjoint(::Vector{Float64}) with eltype Float64:
  0.0199472331085319

: x.value .* 1000
: 3x1 Matrix{Float64}:
  77.40709795031047
  116.06771003983766
  806.5276266877964
```

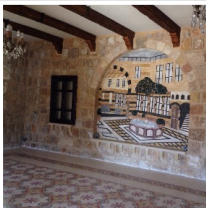
Рис. 18: Вывод результатов

Предположим есть изображение, на котором были изменены некоторые пиксели. Требуется восстановить неизвестные пиксели путём решения задачи оптимизации. Будем использовать пакет **Convex.jl** и оптимизатор (решатель) **SCS**, пакет **ImageMagick.jl** для работы с изображениями.

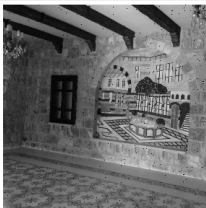
```
#  
import Pkg  
Pkg.add("ImageMagick")  
Pkg.add("Convex")  
Pkg.add("SCS")  
using Images  
using Convex  
using SCS
```

Восстановление изображения

```
# Считывание исходного изображения:  
Kref = load("Downloads/khiam-small.jpg")
```



```
K = copy(Kref)  
p = prod(size(K))  
missingids = rand(1:p,400)  
K[missingids] = RGBX{Nof8}(0.0,0.0,0.0)  
K  
Gray.(K)
```



Восстановление изображения

```
# Матрица цветов:
Y = Float64.(Gray(K));

correctids = findall(Y[:,!0])
X = Convex.Variable(size(Y))
problem = minimize(nuclearnorm(X))
problem.constraints += X[correctids]==Y[correctids]

1-element Vector{Constraint}:
 -- constraint (affine)
  └─ + (affine; real)
      └─ index (affine; real)
          └─ 283x283 real variable (id: 127..095)
              └─ 79690x1 Matrix{Float64}
```

```
solve!(problem, SCS.Optimizer)
```

```
-----
              SCS v3.2.9 - Splitting Conic Solver
              (c) Brendan O'Donoghue, Stanford University, 2012
-----
problem:  variables n: 80090, constraints m: 159780
cones:    z: primal zero / dual free vars: 79690
          nuc: nuclear vars: 80090, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 159780, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
   0 | 1.27e+02 | 1.17e+01 | 3.09e+03 | 1.53e+03 | 1.00e-01 | 3.89e-01
 150 | 4.32e-03 | 6.54e-05 | 1.21e-04 | 4.45e+02 | 9.31e-03 | 1.57e+01
-----
status:  solved
timings: total: 1.57e+01s = setup: 2.29e-01s + solve: 1.55e+01s
          lin-sys: 7.86e-01s, cones: 1.43e+01s, accel: 6.20e-02s
```

Восстановление изображения

```
: @show norm(float.(Gray.(Kref))-X.value)  
@show norm(-X.value)  
colorview(Gray, X.value)
```

```
norm(float.(Gray.(Kref)) - X.value) = 1.129468725543763  
norm(-(X.value)) = 124.33326959946373
```



Рис. 21: Норма и исправленное изображение

Задания для самостоятельного выполнения

1. Решите задачу линейного программирования

$$x_1 + 2x_2 + 5x_3 \rightarrow \max,$$

при заданных ограничениях:

$$-x_1 + x_2 + 3x_3 \leq -5, \quad x_1 + 3x_2 - 7x_3 \leq 10, \quad 0 \leq x_1 \leq 10, \quad x_2 \geq 0, \quad x_3 \geq 0.$$

Задания для самостоятельного выполнения

```
: # 1
# Создание модели
model = Model(GLPK.Optimizer)

# Определение переменных
@variable(model, 0 <= x1 <= 10)
@variable(model, x2 >= 0)
@variable(model, x3 >= 0)

# Определение ограничений
@constraint(model, -x1 + x2 + 3x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10)

# Определение целевой функции (максимизация)
@objective(model, Max, x1 + 2x2 + 5x3)

# Решение задачи
optimize!(model)

# Вывод результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных:")
println("x1 = ", value(x1))
println("x2 = ", value(x2))
println("x3 = ", value(x3))
```

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.0625
Оптимальные значения переменных:
x1 = 10.0
x2 = 2.1875
x3 = 0.9375

Рис. 22: Решение задачи № 1

2. Решите предыдущее задание, используя массивы вместо скалярных переменных

Рекомендация. Запишите систему ограничений в виде $A\vec{x} = \vec{b}$, а целевую функцию как $c^T \vec{x}$.

Задания для самостоятельного выполнения

```
# 2
# Определение данных
c = [1, 2, 5] # Коэффициенты целевой функции
A = [-1 1 3; 1 3 -7] # Матрица ограничений
b = [-5, 10] # Правые части ограничений
lb = [0, 0, 0] # Нижние границы
ub = [10, Inf, Inf] # Верхние границы

# Создание модели
model = Model(GLPK.Optimizer)

# Определение переменных с границами
@variable(model, x[1:3])
for i in 1:3
    set_lower_bound(x[i], lb[i])
    if ub[i] < Inf
        set_upper_bound(x[i], ub[i])
    end
end

# Определение ограничений
@constraint(model, constraints[j in 1:2], sum(A[j, i] * x[i] for i in 1:3) <= b[j])

# Определение целевой функции
@objective(model, Max, sum(c[i] * x[i] for i in 1:3))

# Решение задачи
optimize!(model)

# Вывод результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции: ", objective_value(model))
println("Оптимальные значения переменных:")
for i in 1:3
    println("x$i = ", value(x[i]))
end

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 19.0625
Оптимальные значения переменных:
x1 = 10.0
x2 = 2.1875
x3 = 0.9375
```

Рис. 23: Решение задачи № 2

Задания для самостоятельного выполнения

3. Решите задачу оптимизации

$$\|A\vec{x} - \vec{b}\|_2^2 \rightarrow \min$$

при заданных ограничениях:

$$\vec{x} \geq 0,$$

где $\vec{x} \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $\vec{b} \in \mathbb{R}^m$.

Матрицу A и вектор $\vec{b}$ задайте случайным образом. Для решения задачи используйте пакет **Convex** и решатель **SCS**.

Задания для самостоятельного выполнения

```
# 3
# Задача размер
m = 10
n = 5

# Генерация случайной матрицы A и вектора b
# Для воспроизводимости можно установить seed через Random, но Convex не требует этого
A = randn(m, n)
b = randn(m)

# Определение переменной оптимизации (вектор длины n)
x = Variable(n)

# Целевая функция: минимизация квадрата евклидовой нормы
objective = sumsquares(A * x - b)

# Ограничение: x >= 0 (Все компоненты неотрицательны)
constraints = [x >= 0]

# Формулировка задачи
problem = minimize(objective, constraints)

# Решение задачи с использованием SCS
solve!(problem, SCS.Optimizer; silent_solver = true)

# Вывод результата
println("Оптимальное значение x:")
println(x.value)

println("Минимальное значение целевой функции:")
println(problem.optval)
```

Оптимальное значение x:
[0.825814896884161812; 0.21866317249882198; 0.08812957944875598222; 7.283721288778569e-5; 6.288978223987359e-5;]
Минимальное значение целевой функции:
8.818288224853998

Рис. 24: Решение задачи № 3

Задания для самостоятельного выполнения

4. Проводится конференция с 5 разными секциями. Забронировано 5 залов различной вместимости: в каждом зале не должно быть меньше 180 и больше 250 человек, а на третьей секции активность подразумевает, что должно быть точно 220 человек. В заявке участник указывает приоритет посещения секции: 1 — максимальный приоритет, 3 — минимальный, а значение 10000 означает, что человек не пойдёт на эту секцию. Организаторам удалось собрать 1000 заявок с указанием приоритета посещения трёх секций. Необходимо дать рекомендацию слушателю, на какую же секцию ему пойти, чтобы хватило места всем. Для решения задачи используйте пакет **Convex** и решатель **GLPK**. Приоритеты по слушателям распределите случайным образом

Решение в виде полного листинга предоставлено в отчете.

Задания для самостоятельного выполнения

Статус решения: OPTIMAL
Оптимальное значение целевой функции: 1244.0

Распределение участников по секциям:
Секция 1: 180 человек
Секция 2: 190 человек
Секция 3: 220 человек
Секция 4: 200 человек
Секция 5: 210 человек

Пример рекомендаций для первых 5 участников:
Участник 1: Секция 4 (приоритет: 1)
Участник 2: Секция 4 (приоритет: 1)
Участник 3: Секция 1 (приоритет: 1)
Участник 4: Секция 4 (приоритет: 1)
Участник 5: Секция 1 (приоритет: 1)

Рис. 25: Решение задачи № 4

Задания для самостоятельного выполнения

```
# >
# Создание модели
model = Model(GLPK.Optimizer)

# Определение переменных: количество чашек каждого вида кофе
@variable(model, x_raff >= 0) # Количество чашек "Раф кофе"
@variable(model, x_cap >= 0) # Количество чашек "Капучино"

# Цены на кофе
price_raff = 400 # рублей за чашку
price_cap = 300 # рублей за чашку

# Целевая функция: максимизация выручки
@objective(model, Max, price_raff * x_raff + price_cap * x_cap)

# Ограничения по ресурсам
# Зерна: 40г на Раф, 30г на Капучино, всего 500г
@constraint(model, 40x_raff + 30x_cap <= 500)

# Молоко: 140г на Раф, 120г на Капучино, всего 2000г
@constraint(model, 140x_raff + 120x_cap <= 2000)

# Ванильный сахар: 5г на Раф, всего 40г (нужно использовать весь)
@constraint(model, 5x_raff == 40)

# Решение задачи
optimize!(model)

# Вывод результатов
println("Статус решения: ", termination_status(model))
println("Оптимальное значение целевой функции (выручка): ", objective_value(model), " рублей")
println("\nОптимальный план производства:")
println("Количество 'Раф кофе': ", value(x_raff), " чашек")
println("Количество 'Капучино': ", value(x_cap), " чашек")

# Проверка использования ресурсов
println("\nИспользование ресурсов:")
println("Зерна: ", 40*value(x_raff) + 30*value(x_cap), " г из 500 г")
println("Молоко: ", 140*value(x_raff) + 120*value(x_cap), " г из 2000 г")
println("Ванильный сахар: ", 5*value(x_raff), " г из 40 г")

# Расчет выручки от каждого вида кофе
println("\nВыручка по видам кофе:")
println("От 'Раф кофе': ", price_raff * value(x_raff), " рублей")
println("От 'Капучино': ", price_cap * value(x_cap), " рублей")
```

Рис. 26: Решение задачи № 5

Задания для самостоятельного выполнения

```
Статус решения: OPTIMAL
Оптимальное значение целевой функции (выручка): 5000.0 рублей

Оптимальный план производства:
Количество 'Раф кофе': 8.0 чашек
Количество 'Капучино': 6.0 чашек

Использование ресурсов:
Зерна: 500.0 г из 500 г
Молоко: 1840.0 г из 2000 г
Ванильный сахар: 40.0 г из 40 г

Выручка по видам кофе:
От 'Раф кофе': 3200.0 рублей
От 'Капучино': 1800.0 рублей
```

Рис. 27: Решение задачи № 5

Выводы

В результате выполнения данной лабораторной работы я освоила пакеты Julia для решения задач оптимизации.